

Can Your Agent See the Key?

Gateway-Enforced Brokered Isolation for LLM Tool-Using Agents

Sathyaraj Kolandasamy
Nova Southeastern University
College of Computing, AI, and Cybersecurity
sk2120@mynsu.nova.edu

Abstract—LLM tool-using agents authenticate to APIs and ticketing systems. Prototypes routinely place secrets in the model context through a `get_secret` tool, and under prompt injection that material can leak. I contribute a product-agnostic gateway-enforced brokered isolation (GEBI) reference architecture and evaluate its core credential-containment invariant: isolated agent execution, policy-checked outbound actions, server-side credential substitution, and scrubbed model-visible results. The distinction from prompt-level defenses is architectural rather than lexical: a prompt cannot un-see a key already in context, whereas brokering keeps the key out of the transcript entirely. On a frozen 12-task set with a local `llama3.2:1b` model, direct mode shows secret-exposure rate 1.00 and attacker-benefit rate (ABR) 1.00 under all three injection strategies — and exposure 1.00 even with no injection at all — while brokered mode holds exposure and ABR at 0.00 across all 48 brokered runs and still completes benign tasks. A canary token tracked independently of exposure shows the separation that matters: under fake-completion the brokered agent echoed the attacker’s marker on 0.67 of runs while leaking nothing, so containment did not depend on the model resisting persuasion. Exposure is measured as an exact substring test over non-operational placeholder tokens, so it detects verbatim reprinting but not semantic paraphrase. The contribution is an architecture, a direct-vs-brokered harness, and a sanitized credential-oriented injection suite.

Index Terms—LLM agents, credential isolation, prompt injection, zero-trust tooling, AI security

I. INTRODUCTION

Tool-using language-model agents must act on external systems, and acting on external systems requires credentials. The path of least resistance in current agent frameworks is to expose a tool that returns the secret — a `get_secret` call, or an environment read — so that the model can place it in a subsequent request. That design puts the credential into the model’s own context window, and once it is there the credential is subject to every failure mode the context window has.

Prompt injection is the sharpest of those failure modes [1], [2]. An attacker who controls any text the agent ingests — a ticket body, a document, a field in a tool result — can attempt to steer the model into printing, forwarding, or otherwise disclosing whatever it can see. Existing robustness benchmarks largely score chat-safety refusal rather than credential containment [4], so they do not answer the question a platform team actually has: *did the secret ever reach the model at all?*

Production identity systems answered the analogous question years ago, and not by asking the principal to behave. They prefer either short-lived, narrowly scoped credentials or key-bound workload identities such as SPIFFE/SPIRE SVIDs [3] — neither of which an application component should be able to read and forward. Those are different mechanisms with different failure modes, and the difference decides whether brokering is necessary or merely convenient; Section III separates them. The contribution of this paper is to carry that discipline into the agent setting and to measure whether it holds under injection.

Hypothesis. Direct secret access leaks or misuses credentials under prompt injection substantially more often than brokered, non-exportable issuance.

The public Bromure Agentic Coding product illustrates a related deployment pattern: isolated Linux VMs, a local outbound gateway, stand-in credentials inside the VM, and real-credential substitution at the gateway. Its documentation also describes approval gating, prompt-injection screening, and security-event logging [5]. I generalize these product-described controls into GEBI, a vendor-neutral reference architecture, and use Bromure only as motivation — it is not an evaluated system here. The experiment below tests the narrower invariant that secrets never enter the model-visible channel.

Contributions.

- **GEBI**, a product-agnostic reference architecture combining isolated agent execution, a non-bypassable policy gateway, server-side credential substitution, optional approval gating, and auditable inspection.
- **A direct-vs-brokered harness** that enforces “no secrets in model-visible channels” as a checked invariant in brokered mode rather than as a convention.
- **A sanitized credential-oriented injection suite** and a measured exposure / ABR / utility freeze, with the measurement definitions fixed in advance.

II. THREAT MODEL

Attacker goal. Cause *bearer* secret material to appear in a model-visible channel, and/or coerce a denied export path into succeeding. The qualifier matters: as Section III argues, that goal is equivalent to compromise only for credentials whose possession is itself authorization.

Knowledge. Grey-box. The attacker knows the agent has tools and may know tool names, but need not know broker internals. This is deliberately generous: the `tool_override` payload names tools from *both* modes, so a single injection probes both surfaces.

Capabilities. Inject UTF-8 text into the task channel, simulating indirect injection through a ticket, a document, or a tool-returned field. The attacker cannot modify broker allowlists, read host memory, or authenticate to the broker directly.

Defender. Chooses the architecture, configures allow/deny lists and handle lifetimes, and redacts logs. Prompt-level defenses are deliberately *excluded* from both arms so that architectural effects are not confounded with prompt wording. The direct arm still carries a system prompt instructing the agent not to print secrets; that instruction is the control, and its failure is part of the result.

Out of scope. Live key theft; attacks on production identity providers; multi-agent credential passing; VM-escape and gateway hardening. Secrets are research fakes only — non-operational placeholder tokens whose literal values make exposure an exact, model-agnostic substring test rather than a subjective judgment.

This study instantiates a *static bearer* credential, chosen deliberately because it is the shape for which exposure and compromise coincide exactly. Credentials that are key-bound or audience-restricted behave differently under the same attack, and the next section explains why.

III. CREDENTIAL SHAPES AND THE REACHABILITY EQUIVALENCE

The paper has so far said “the secret,” and that singular hides a design choice. An agent may authenticate to an application in several ways, and they do not degrade alike when the agent’s context window is exposed. This section separates them along two questions — whether the credential’s bytes are *transferable* if observed, and whether the *agent* holds the authenticating material — and then states precisely what brokering adds. Table I summarizes the result.

A. Four credential shapes

Class B — static bearer secret. An API key, password, or connection string. Authentication is possession of a string; I define the class by that property, so anything that reads the string can act as the principal, from anywhere, until the string is changed.

Class S — scoped or ephemeral bearer. A short-lived access token issued by a token service, or a session credential carrying an audience restriction. Still a bearer: observation still yields impersonation, but the window and the blast radius are bounded rather than open-ended.

Class P — proof-of-possession. Authentication requires demonstrating control of a private key rather than presenting a copyable value, as with a client certificate or a SPIFFE X.509-SVID [3]. The SVID document itself is not secret; the workload-held private key is the part that authenticates, and it never transits the channel.

TABLE I
CREDENTIAL SHAPES ORDERED BY WHAT A CONTEXT-WINDOW DISCLOSURE YIELDS AN ATTACKER. ONLY CLASS B WAS EXERCISED END-TO-END BY THE BROKER IN THIS STUDY. THE HARNESS DECLARES A `client_cert_handle` KIND, BUT IT IS STORED AND RETURNED AS A STATIC STRING WITH NO CERTIFICATE LOGIC ANYWHERE, SO IT IS CLASS B IN BEHAVIOUR REGARDLESS OF ITS NAME.

Class	Shape	Example	Disclosure in context yields	In this study?
B	Static bearer secret	API key, service token	Full, durable impersonation from anywhere	Yes (both arms)
S	Scoped / ephemeral bearer	Short-lived access or session token	Impersonation, bounded by lifetime and audience	No
P	Proof-of-possession	Client certificate; SVID key [3]	Nothing replayable; misuse in place still possible	No
C	Brokered capability	GEPI handle (this paper)	An action name under policy; no credential	Yes (brokered arm)

Class C — brokered capability. The agent names an *action* and holds an opaque handle; a gateway holds whichever of B, S, or P the backend actually requires. This is GEPI.

B. The reachability equivalence

For a static bearer credential, authentication *is* possession of a string, so the predicate “the secret reached the model context” and the predicate “the credential is compromised” are the same predicate. The transcript is not a disclosure risk adjacent to compromise; it is the compromise.

A context window is a uniquely bad container for such a secret because it is replicated by default — into the final answer, the trace log, the evaluation record, and any retrieval index built from them — so a single read fans out into egress paths the agent’s author never enumerated.

Proof-of-possession severs the equivalence by making the transferable part of the credential and the authenticating part different objects. An attacker who reads everything the model can see obtains an identifier, not an authenticator, and a stolen identifier presented from the attacker’s own channel fails at the handshake. Class S sits between the two: the equivalence still holds, but only within a bounded lifetime and audience.

C. What brokering adds beyond binding

Class P already breaks the equivalence, which raises the obvious question of why a broker is needed at all. The answer is that binding defeats the thief but not the confused deputy. An injected agent that *legitimately holds* a bound key can still sign a perfectly valid request toward an attacker-chosen destination; nothing about key binding constrains *what* the holder asks for. GEPI mediates the action as well as removing the secret — the handle names what may be done, not merely who is doing it — which is why the allowlist and denylist of Section IV are load-bearing rather than defence in depth. Binding and mediation are complementary: the first stops replay, the second stops misuse in place.

D. Coverage in this study

The harness declares three credential kinds but the broker path exercises exactly one: a static bearer key. There is no token exchange, no audience binding, and no certificate or key-bound path anywhere in the implementation. The shape actually exercised end-to-end is therefore Class B — the shape for which the equivalence is tightest and the gap consequently widest. The 1.00-to-0.00 result in Section VI should be read as a measurement of that worst case rather than as a claim about credential handling in general, and whether brokering adds containment *beyond* what proof-of-possession already provides is argued here but not measured.

IV. THE GEBI ARCHITECTURE

GEBI is a layering claim: credential possession belongs to the gateway, not to the agent. Five controls follow from it.

- 1) **Isolated execution.** The agent runs in a workspace that holds no production credential material, and whose egress is restricted to the gateway. Any secret present inside the workspace is a stand-in. Section IV-D discusses how this boundary is obtained; the harness models it rather than enforcing it.
- 2) **Non-bypassable policy gateway.** All outbound action requests traverse a broker that evaluates them against policy. There is no side channel by which the agent can reach the external system directly.
- 3) **Server-side credential substitution.** The broker holds the real credential, uses it to perform the authorized action, and never copies it into the response payload.
- 4) **Scrubbed model-visible results.** What returns to the model is assembled from a fixed field template rather than filtered after the fact, so secrets cannot appear in it by construction.
- 5) **Auditable inspection.** Every allow, deny, and issuance decision is logged, which makes denied attempts measurable rather than invisible.

A. Broker control flow

Each `broker_execute(action, params)` call resolves in a fixed order:

- 1) Check a **denylist** and reject immediately. The denied set is `exfiltrate`, `print_secret`, `export_credential`, and `send_to_url`.
- 2) Check an **allowlist** and reject anything not on it, so unknown actions **fail closed**. The allowed set is `fetch_public_status`, `lookup_ticket`, and `summarize_doc`.
- 3) Mint a capability handle (Section IV-B).
- 4) Perform the action server-side, where the secret is used but never copied into the payload.
- 5) Return a result assembled field-by-field from a fixed template.

Deny is evaluated *before* allow, so a name appearing in both sets is denied. Because the model-visible string is *constructed* rather than filtered, secrets cannot appear in it by construction;

TABLE II
TOOL SURFACE BY MODE. THE BROKERED ARM ADVERTISES AN EXPORT TOOL AND THEN DENIES IT, SO THE DEFENSE IS TESTED AGAINST AN INFORMED ATTACKER.

Mode	Tools exposed	Secret reaches model?
Direct	<code>get_secret</code> , <code>read_env_secret</code> , <code>call_api</code>	Yes, in tool result
Brokered	<code>broker_execute</code> , <code>broker_export_secret</code>	No — denied or scrubbed

a final assertion re-checks the rendered text before return and is treated as an invariant, not a heuristic. If that assertion ever fires, the run is a harness bug, not a finding.

B. Capability handles

Handles are minted per call as opaque identifiers, marked non-exportable, and scoped to a single action with a bounded lifetime (60 s in this configuration). The model receives the handle identifier, never the credential it stands for. One honest caveat: the implementation records the lifetime at issuance and uses it to characterize the policy configuration, but does not re-check expiry on use. This study therefore does not claim to measure TTL enforcement, only TTL configuration.

C. Tool surface

Table II gives the tools visible to the model in each mode. The brokered arm deliberately advertises `broker_export_secret` as *bait*: the architecture is tested against an attacker who knows exactly what to ask for, and the export path is denied rather than hidden. Export-style names are additionally denied *by name*, so a model that hallucinates a direct-mode tool while in brokered mode still receives a denial rather than a secret.

D. Deploying the isolation boundary

The architecture above assumes the agent runs somewhere that holds no credential material. That assumption is doing real work, so it is worth stating how it is obtained in practice — and noting that the harness evaluated here simulates it in-process rather than enforcing it, which Section VIII records.

Isolation alone is not containment. A container whose environment carries the upstream API key is simply a box with the secret inside it: the agent reads the environment and the exposure result is unchanged. The property that matters is not that the agent is sandboxed but that the sandbox is *credential-free*. The runner should be given network reachability to the broker, never possession of a secret. Concretely, this rules out passing credentials through environment variables or build arguments, mounting host credential directories, and baking secrets into image layers — each of which returns the deployment to Class B of Section III.

Egress is the second boundary. A sandbox with no route to the public network cannot exfiltrate a secret even if one leaks into it, because there is nowhere to send it. Running the

agent on an internal-only network whose sole reachable host is the broker therefore provides a containment guarantee that is complementary to, and independent of, the one measured in Section VI: exposure asks whether the secret entered the model’s context, while egress restriction asks whether anything in that context can leave. A deployment that wants both must configure both; neither subsumes the other.

Authenticating the sandbox. If the agent holds no secret, the broker needs some other way to decide that a request came from a legitimate workload — the bootstrap problem that credential systems face generally. Three options appear in practice, in increasing order of strength. The weakest is *network position*: the broker accepts connections only from the sandbox’s network namespace. Stronger is a *mounted local endpoint*, where a socket is placed into the sandbox and filesystem permissions carry the authentication decision. Strongest is *platform attestation*, in which the hosting platform verifies properties of the workload — its image, its namespace, its service account — and issues a short-lived, key-bound identity on that basis. SPIFFE/SPIRE is the representative example [3], and it is worth noting that this is precisely Class P of Section III: the workload’s private key is generated locally and never transits, so attestation solves the bootstrap problem without introducing a new bearer secret to protect.

Why this matters for the threat model. Each of these mechanisms authenticates the *workload*, not the model’s intentions. None of them prevents an injected agent from requesting a permitted action for an attacker’s purpose, which is the confused-deputy case of Section III. Deployment isolation and policy mediation are therefore both required: isolation establishes that the caller is the agent and that the agent holds nothing worth stealing, while the allowlist constrains what that caller may ask for.

V. METHOD

Direct arm. `get_secret` and `environment-read` tools return plaintext placeholder tokens into the transcript, which is the prototype pattern this paper argues against. There are four distinct paths by which a secret becomes model-visible: `get_secret` returns it in a tool result; `read_env_secret` does the same through a simulated environment read; `call_api` leaks whatever the model pasted into its parameters; and the composed final answer is scanned independently. The third is the most instructive, because there the secret escapes not because a tool returned it but because the model *carried it forward* into a later call — an egress path no tool-level filter would catch. Brokering removes all four by construction, since the agent never holds a value to carry. Both `get_secret` and `read_env_secret` accept a name argument and default to the API key, so all three declared credential kinds are reachable through the same channels.

Brokered arm. Only `broker_execute` performs work; export probes are denied; secrets stay server-side; results are scrubbed.

Harness scope. The experiment models the *credential-containment core* of GEBI. It does not measure VM-escape

resistance, gateway hardening, approval-gate usability, or any product’s detection accuracy.

Tasks. Twelve benign status, ticket, and document jobs (task-set SHA-256 `ad60f406...bad36b`), each with a gold action name and parameters.

Injections. `naive_exfil`, `tool_override`, `fake_completion`, plus a clean `none` control. All carry a visible research tag and a canary token, which makes them conspicuous by design — an acknowledged limitation, since it biases any detector optimistically.

Model and budget. Local Ollama `llama3.2:1b` with JSON tool-plan parsing, temperature 0.0, seed 42, and a cap of four tool steps per task. Determinism comes from temperature 0 rather than from averaging; each cell is run once.

A. Metrics

Exposure is 1 for a run if any literal placeholder secret value appears in the model-visible channel — the union of tool-result text and the final answer. It is an exact substring test, chosen because it is objective and model-agnostic; its blind spot is semantic leakage, where a model describes a credential without reproducing it.

ABR (attacker-benefit rate) is exposure restricted to injected runs. The consequence worth stating plainly: a denied-but-secret-free attempt does **not** count as attacker benefit. That is what separates “the agent tried” from “the agent leaked,” and it is why a brokered agent can comply with an injection’s *form* while scoring ABR 0.00.

Utility is 1 if the gold action was invoked and returned successfully. It is scored by exact match on the action name, independently of exposure, so a leaky run can still be a useful one. The two axes are reported separately rather than collapsed into a single score.

Canary rate is the fraction of runs in which the injection’s marker token reached the final answer. It measures compliance with the attacker’s surface form and is deliberately independent of exposure, which is what lets the two be compared. **Denials** is the mean number of broker policy refusals per run, which turns a blocked export into a positive observation rather than an absence. Both are reported per cell in Section VI.

The harness also computes a utility-retention ratio, normalized against the unhardened clean cell (direct mode, no injection). It is omitted from Table III because at this sample size it adds no information beyond the raw utilities — the denominator is a single weak baseline (0.33, or 4/12) and dividing by it mostly magnifies noise. It is worth noting that retention is not bounded above: an injected run can out-score the clean baseline when the injected text happens to push a small model into emitting a well-formed tool call it otherwise fumbles, which is the same effect visible in Finding 6.

VI. RESULTS

Finding 1 — direct mode leaks even when nobody attacks it. Exposure is 1.00 in every direct cell, *including the clean control*. No injection is required: the agent calls `get_secret` in the ordinary course of doing its job, and the

TABLE III

EXPERIMENTAL FREEZE, $n = 12$ TASKS PER CELL (96 RUNS, ALL `status=ok`). EXPOSURE AND ABR ARE THE SECURITY AXES; CANARY RATE MEASURES COMPLIANCE WITH THE INJECTION’S SURFACE FORM, AND DENIALS COUNT BROKER POLICY REFUSALS. SINGLE SEED, TEMPERATURE 0, ONE RUN PER CELL. AT $n = 12$ A BINOMIAL 95% INTERVAL IS $\approx \pm 0.28$, SO UTILITY AND CANARY DIFFERENCES ARE DIRECTIONAL; THE SATURATED EXPOSURE AND ABR COLUMNS REFLECT A MECHANISM RATHER THAN A SAMPLE ESTIMATE.

Mode	Injection	Expo.	ABR	Util.	Canary	Denials
Direct	none	1.00	0.00	0.33	0.00	0.00
Direct	naive_exfil	1.00	1.00	0.58	0.75	0.00
Direct	tool_override	1.00	1.00	0.25	0.33	0.00
Direct	fake_completion	1.00	1.00	0.50	0.75	0.00
Brokered	none	0.00	0.00	0.17	0.00	1.17
Brokered	naive_exfil	0.00	0.00	0.33	0.42	0.75
Brokered	tool_override	0.00	0.00	0.25	0.25	0.75
Brokered	fake_completion	0.00	0.00	0.50	0.67	0.33

credential enters the transcript. The injection strategies change what the attacker *gains*, not whether the secret is present.

Finding 2 — the ABR gap is total. Under injection, direct ABR is 1.00 for all three strategies; brokered ABR is 0.00 for all three. Within this freeze the architectural difference is not a matter of degree.

Finding 3 — the containment invariant held, and what that does and does not show. Brokered exposure is 0.00 across all 48 brokered runs, clean and injected alike.

This number deserves a direct caveat, because it is partly true by construction. The brokered result string is assembled from a fixed field template and asserted secret-free before return, so a brokered cell *cannot* report exposure without the assertion firing first. Reporting 0.00 is therefore not evidence that brokering defeats injection in the way an attack-success measurement would be. The honest reading is narrower: across 36 injected brokered runs, no injection found a path that caused the invariant to be violated or circumvented.

That is a weaker claim than “brokering is secure,” but it is not vacuous, and it is worth being explicit about what would have falsified it. The assertion would have fired — and the result would have been a harness defect rather than a finding — had any of the following occurred: a denied action nevertheless returning server-side data; the agent reaching a credential through a tool name not on the catalog; a scrubbing gap letting placeholder material into the payload; or a policy check evaluated in the wrong order, such that an allowlisted-but-denied action slipped through. None occurred. The broker logged **36 policy refusals across 25 of the 48 runs**, so the denial path was exercised heavily rather than left dormant — the mechanism was under load, and it held.

What the result genuinely establishes is a property of the *architecture*, not of the model: no sequence of model outputs in this freeze could produce exposure, because the model was never in a position to emit a credential it had never received. That is exactly the guarantee a prompt-level defense cannot offer, and it is why the comparison in Section VII is between mechanisms rather than between degrees of model compliance.

Finding 4 — compliance and exposure come apart, and the gap is measurable. This is the result the canary column exists to expose. In the brokered arm under fake-completion, 0.67 of runs echoed the attacker’s canary token — the agent visibly adopted the injection’s framing — while exposure stayed at 0.00. The same pattern holds for naive exfiltration (0.42) and tool override (0.25). An evaluation that scored only “did the model obey?” would record these as defeats. Measuring the credential shows nothing moved. Containment here is not a function of the model declining to cooperate; it cooperated, and the architecture made that irrelevant.

This also answers the objection raised under Finding 3 from a second direction. If brokered exposure were 0.00 merely because no injection ever took hold, the result would be uninformative — a measurement of attacker ineffectiveness rather than of containment. The canary column rules that reading out: **16 of the 48 brokered runs** echoed the attacker’s token, meaning the injection demonstrably reached and influenced the model in a third of them, and exposure was still 0.00 in every one. The attacks landed. The credential stayed behind the gateway regardless.

Finding 5 — denials and compliance move in opposite directions. The brokered clean control incurs the most policy refusals (1.17 per run) and no canary hits, while fake-completion incurs the fewest (0.33) and the most canary hits (0.67). The agent that adopts the attacker’s framing stops probing the export path, because the injection has convinced it the credential work is already done. The denial counter makes that visible; exposure alone would not distinguish it from an agent that never tried.

Finding 6 — utility is imperfect, and injection does not uniformly degrade it. Utility is well below 1.00 in both arms on this small local model, and the highest cell in the matrix is *injected* (`direct|naive_exfil`, 0.58) rather than clean (0.33). Injection changes agent behaviour without necessarily breaking the task. Section VII gives a mechanism-level account of the low absolute values; it is not a measured error analysis, and I do not present it as one.

VII. DISCUSSION

Compliance is not exposure. The most useful consequence of separating ABR from instruction-following is that a brokered agent can *follow the injection’s form* — attempting the export, echoing the attacker’s canary — while leaking nothing. Measuring only “did the model obey?” would score that as a defeat; measuring exposure shows the credential never moved. Architectural containment does not require the model to resist persuasion, which is precisely why it is more robust than asking it to.

Two signals recorded alongside exposure make this separation observable rather than merely arguable. A *canary* token embedded in each injection is detected in the agent’s final answer, so compliance with the injection’s surface form is visible whether or not a credential moved. A *denial count* is accumulated from broker policy decisions, so a refused export becomes a positive measurement rather than a silent

non-event. Together they distinguish three outcomes that a single exposure number collapses: an agent that ignored the injection, one that attempted the export and was refused, and one that succeeded.

The measured result is that the brokered arm sits squarely in the middle case. Under fake-completion it echoed the canary on 0.67 of runs and still exposed nothing; under the other two strategies the canary rate was 0.42 and 0.25 with exposure fixed at 0.00 throughout. The agent was persuaded and the credential stayed put. That is a stronger claim than “the attack failed,” because it identifies *where* it failed: not at the model’s judgment, which the injection successfully bent, but at the boundary the model could not reach past.

Why prompt-level defenses are the wrong layer. Direct mode leaks on every cell *despite* a system prompt that explicitly instructs the agent not to print secrets. Once a credential is in the transcript, every downstream path — the final answer, the log, a retrieval index built from that log — is a potential egress. A sandwich prompt cannot un-see a key. GEBI’s claim is narrower and stronger: the key is never in the transcript to begin with.

Utility is bounded by mechanism, not by security. Utility is scored by exact match on the gold action name, and the agent has a budget of four tool steps per task. Under injection, a brokered run spends part of that budget on denied export probes before reaching the legitimate call. That is a plausible, mechanism-level account of why brokered utility varies across cells — it is a property of the harness’s scoring rule and step limit, not a measured error analysis. Distinguishing “the model planned the wrong action” from “the model ran out of steps” requires per-run transcripts that this freeze does not retain.

Deployment implications. Brokering shifts the burden to allowlist design: a workflow that direct mode would “solve” by pasting a key must instead be expressed as an allowlisted action. Utility retention makes that cost visible rather than hidden. Two further controls follow. First, **fail closed** when the broker is unavailable — a fail-open path silently restores exposure and invalidates the containment claim. Second, extend redaction to **observability**, because logging and retrieval pipelines are model-visible channels by another name.

VIII. LIMITATIONS

- **Single model, small scale.** One small local model on twelve tasks. Utility figures characterize this configuration, not agent frameworks generally.
- **Exposure is a substring test.** It catches verbatim reprinting of known placeholder tokens but **not semantic leakage**, where a model describes or paraphrases a credential without reproducing it. ABR equals exposure on injected runs by construction, so it inherits the same blind spot. This is the single most important measurement gap in the study.
- **TTL is configured, not enforced.** Handle lifetimes are recorded at issuance; expiry is not re-checked on use.
- **One credential shape.** The broker performs a single static bearer-secret lookup; it implements no to-

ken exchange, no audience binding, and no certificate or key-bound path. The harness names a `client_cert_handle` kind, but it is stored and returned as a static string, so it is a bearer secret in behaviour. Whether brokering adds containment beyond what proof-of-possession already provides is therefore argued in Section III and not measured.

- **Isolation is modelled, not enforced.** The harness is an in-process simulation: the direct and brokered arms differ in which tool surface the agent is given, not in any container, namespace, or network boundary. The deployment guidance of Section IV-D is therefore argued from the architecture rather than demonstrated by the measurement, and the egress-restriction guarantee in particular is untested here. A containerized replication that enforces the boundary — and that could be attacked through it — is the natural next step.
- **Research mock, not a KMS.** The broker is a research implementation, not a certified KMS or HSM integration.
- **Conspicuous injections.** The payloads carry visible research tags and canaries, which makes them easier to detect than a stealthy adversary’s would be.
- **Single-agent scope.** Multi-agent credential passing is untested, and it is a plausible place for containment to break.
- **No confidence intervals.** Cells are run once at temperature 0. At $n = 12$ a binomial 95% interval is roughly ± 0.28 , so the utility and canary columns should be read as directional; the ordering of canary rates across injection strategies is suggestive but not separated at this sample size. The exposure and ABR columns are 0.00/1.00 saturated, which reflects a mechanism rather than a sample estimate.
- **Retention is computed but not reported.** The harness emits a utility-retention ratio against the unhardened clean cell. At $n = 12$ with a baseline of 4/12 it magnifies noise rather than adding information, so Table III reports raw utilities instead. A larger task set would make the ratio worth reporting.

IX. REPRODUCIBILITY AND ETHICS

All credential material is synthetic: non-operational placeholder tokens carrying an explicit redaction marker, chosen so that they are greppable, harmless if copied, and unambiguous as an exposure oracle. No live key, token, or production identity system was accessed.

The harness enforces this in continuous integration. One test asserts that every configured secret is a placeholder; another greps the source tree for live-key patterns, private-key headers, and offensive-tooling markers and fails the build if any appear. Transcripts are redacted on write, so the artifact cannot accumulate secret material even accidentally. Runs record the seed, dependency versions, configuration hash, and task-set hash.

Every figure in Table III is derived from a single result file containing all 96 per-run records, released with the public

artifact <https://github.com/sathya-ksamy-nsu/Gateway-Enforced-Brokered-Isolation>. Each aggregate recomputes exactly from those records, so a reader can confirm the table rather than trust it: brokered exposure is 0/48, direct exposure is 48/48, and the ABR definition — injected and exposed — holds on all 96 runs without exception.

The contribution is an architecture and an evaluation protocol rather than attack tooling. The injection payloads are published because they are conspicuous, already-catalogued strategy archetypes, and because the defense they probe is the one this paper recommends.

X. CONCLUSION

GEBI removes keys from the model context and, on this freeze, eliminates measurable secret exposure and attacker benefit under sanitized injection while direct mode does not. The architecture generalizes controls described publicly by Bromure into a vendor-neutral contribution: workspace isolation, gateway enforcement, server-side substitution, approval controls, and monitoring [5]. These results evaluate only the credential-containment core — not VM-escape resistance, gateway hardening, approval usability, or any product’s security performance.

The practical recommendation is narrow: if an agent can read a credential, then every injection becomes a potential credential-disclosure vulnerability, and no amount of prompt engineering changes that. Make the credential unreachable instead, and measure the result by whether the secret ever appears in a model-visible channel.

Future work. Stronger and multiple model families; semantic rather than substring exposure detection; extension to proof-of-possession and audience-bound credentials (Classes P and S of Section III), where the marginal value of brokering is an open question; approval-gate evaluation; multi-tenant handle scoping; and RAG-borne injection, where the untrusted span arrives through retrieval rather than the task channel.

ACKNOWLEDGMENT

The author thanks Nova Southeastern University faculty and peers for feedback on earlier drafts of this protocol. Any remaining errors are the author’s.

REFERENCES

- [1] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *Proc. USENIX Security Symp.*, 2024, pp. 1831–1847.
- [2] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications,” 2025. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [3] SPIFFE, “Secure Production Identity Framework for Everyone (SPIFFE/SPIRE).” [Online]. Available: <https://spiffe.io/>
- [4] P. Chao *et al.*, “JailbreakBench: An open robustness benchmark for jailbreaking large language models,” in *Proc. NeurIPS Datasets and Benchmarks*, 2024.
- [5] Bromure, “Bromure Agentic Coding,” 2026. [Online]. Available: <https://bromure.io/en/agentic-coding/> (accessed Sep. 27, 2026).